

Cache Replacement Policy Comparison.

CS 4200 -
Computer
Architecture

Introduction

01

- A cache is a small, fast memory that sits between the CPU and main memory

02

- When the cache is full, a replacement policy decides which block, or part, to evict

03

- Three policies compared: LRU, FIFO, and Random

04

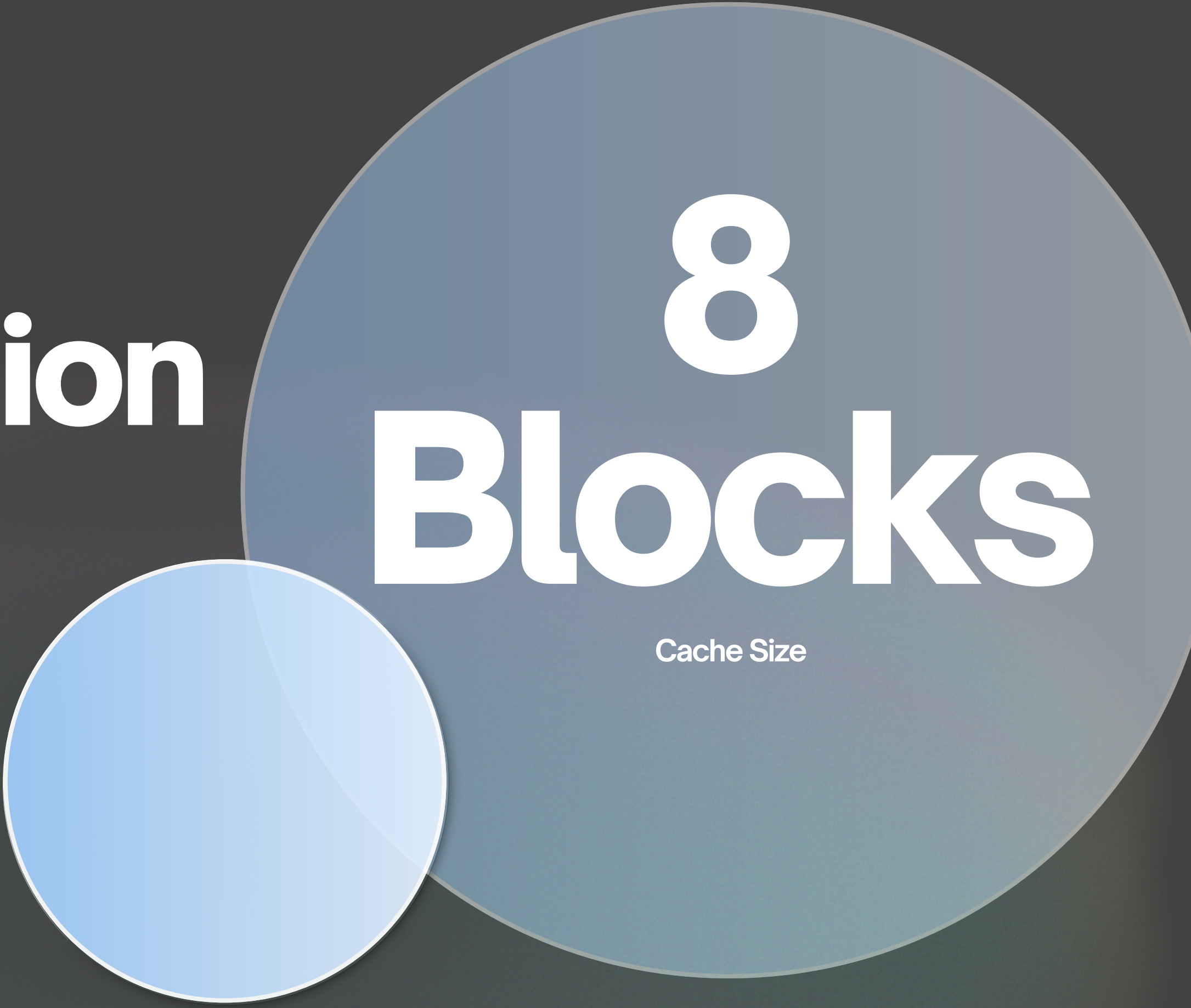
- Goal: measure how each policy preforms across different memory access patterns

Code Structure

- One base Cache class: tracks, hits, misses, and evictions
- Three subclasses, one per policy: LRU Cache, FIFO Cache, and Random Cache
- Four memory trace types: Sequential, Looping, Random, Mixed
- LRU core logic

```
def access(self, address):
    if address in self.cache:
        #Hit: Move the accessed address to the end to mark it as most recently used
        self.cache.move_to_end(address)
        self.hits += 1
    else:
        #Miss: Add the new address to the cache
        self.misses += 1
        if len(self.cache) >= self.cache_size:
            #Evict the least recently used item (the first item in the OrderedDict)
            self.cache.popitem(last=False)
            self.evictions += 1
        self.cache[address] = True
```


Code Demonstration



8
Blocks

Cache Size

- Running all three policies across 4 memory trace types
- Cache size: 8 blocks
 - Address space: 32 | 200 accesses per trace
- Watch for: hit rate differences between policies on the mixed trace